Project - High Level Design

on

Sports Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	AANCHAL MARATHE	EN22CS301015
02	AAHMAD BURHANI	EN22CS301010
03	ABHISHEK SANKHALA	EN22CS301035
04	YASHASVI MODI	EN22CS3011114
05	AKSHAT YADAV	EN22CS301095

Group Name: 10D1

Project Number: AAI-10

Industry Mentor Name:

University Mentor Name: Nishant Shrivastava

Academic Year: 2025-2026

Table of Contents

1. Introduction.
   - 1.1. Scope of the document.
   - 1.2. Intended Audience
   - 1.3. System overview.
2. System Design.
   - 2.1. Application Design
   - 2.2. Process Flow.
   - 2.3. Information Flow.
   - 2.4. Components Design
   - 2.5. Key Design Considerations
   - 2.6. API Catalogue.
3. Data Design
   - 3.1. Data Model
   - 3.2. Data Access Mechanism
   - 3.3. Data Retention Policies
   - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   - 7.1 Security Aspects
   - 7.2 Performance Aspects
8. References

1. Introduction

The Sports Planning Assistant Agent is an advanced AI-driven system designed to assist users in analyzing, planning, and generating insights for various sports scenarios. The system leverages modern concepts from Agentic AI, enabling autonomous decision-making through multiple intelligent agents working collaboratively.

The application accepts high-level user goals in natural language (such as performance analysis, training plan creation, or match statistics) and processes them using a structured reasoning pipeline. It automatically identifies relevant entities such as sport type, league, teams, or players, retrieves real-time data from external APIs, and produces meaningful outputs including structured insights and visual analytics.

The system integrates technologies such as LangChain and CrewAI to implement a modular and scalable architecture. A user-friendly interface is developed using Streamlit to ensure accessibility and ease of interaction.

By combining multiple content modalities, the system improves audience engagement, enhances understanding of sports analytics, and accelerates content production workflows.

This document presents the High-Level Design (HLD) of the system, including architecture, workflow, components, and design considerations.

1.1 Scope of the Document

This document provides a High-Level Design (HLD) of the Sports Planning Assistant Agent system. It outlines the overall system architecture, major components, and interactions between different modules.

The scope of this document includes:

- Description of system functionality and architecture
- Overview of multi-agent design and responsibilities
- Explanation of data flow and processing pipeline
- Integration with external sports data APIs
- High-level description of the user interface
- Visualization and analytics capabilities

This document focuses on the design perspective and does not include low-level implementation details such as code structure or algorithmic optimizations.

1.2 Intended Audience

This document is intended for:

- **Faculty and Evaluators** – to understand the system design, architecture, and implementation approach
- **Developers and Team Members** – to gain clarity on system structure and component interactions
- **AI/ML Enthusiasts** – to explore the application of Agentic AI concepts in real-world scenarios
- **Project Reviewers** – to evaluate the technical depth and design quality of the system

The content assumes a basic understanding of software development, artificial intelligence concepts, and system architecture.

1.3 System Overview

The Sports Planning Assistant Agent is built as a multi-agent AI system that follows a structured pipeline to process user goals and generate outputs.
The system workflow is as follows:

1. The user provides a high-level sports-related goal through the interface.
2. The system analyzes the input using intelligent agents to identify key elements such as sport, league, and entities.
3. A planning agent decomposes the goal into actionable steps.
4. A data retrieval agent interacts with external APIs (e.g., TheSportsDB API) to fetch real-time data.
5. An analysis agent processes the retrieved data to generate insights.
6. The final output is presented to the user in structured textual format along with visual elements such as charts.

The system also implements a reasoning mechanism based on the ReAct framework, allowing agents to dynamically decide actions such as tool usage and data retrieval.
Overall, the system demonstrates a modern approach to AI system design by combining:

- Natural Language Understanding
- Multi-agent collaboration
- Real-time data integration
- Interactive visualization

2. System Workflow:

The Sports Planning Assistant Agent follows a structured and intelligent workflow to process user inputs and generate meaningful outputs. The system is designed using a

multi-agent architecture combined with dynamic reasoning, enabling it to handle complex sports-related queries efficiently.

Step 1: User Input

The workflow begins when the user enters a high-level sports-related goal through the interface built using Streamlit.
Examples of user inputs include:

- "Analyze the performance of Los Angeles Lakers in NBA"
- "Create a training plan for a cricket player for IPL season"

This input is passed to the backend system for processing.

Step 2: Goal Analysis and Entity Detection

The input goal is first processed by the Goal Analyzer Agent.
This agent performs:

- Identification of sport type (e.g., cricket, football, basketball)
- Detection of league (e.g., IPL, NBA, Premier League)
- Extraction of teams or players
- Understanding of the user's intent (analysis, planning, statistics, etc.)

This step uses natural language understanding techniques powered by LangChain.

Step 3: Task Planning

After extracting relevant information, the Planner Agent decomposes the goal into structured steps.
For example:

- Identify league and sport context
- Retrieve team or player data
- Analyze performance metrics
- Generate insights or plan

This ensures that complex goals are broken down into manageable tasks.

Step 4: ReAct-Based Reasoning and Tool Selection

The system implements a reasoning loop based on the ReAct framework.
In this phase, the Data Agent performs:

- **Thought**: Determines what information is required
- **Action**: Selects an appropriate tool (API call)
- **Observation**: Processes the retrieved data
- **Iteration**: Repeats until sufficient information is collected

This allows the system to dynamically decide which tools to use instead of following a fixed sequence.

Step 5: Data Retrieval from External APIs

The system fetches real-time sports data using external APIs such as TheSportsDB API. The data retrieved may include:

- Team statistics
- Player information
- League details
- Match performance data

This ensures that the output is **relevant and up-to-date**.

Step 6: Data Analysis and Insight Generation

The Analysis Agent processes the retrieved data to generate meaningful insights. This includes:

- Performance evaluation
- Trend identification
- Strength and weakness analysis
- Strategy or training recommendations

The output is structured into a clear and readable format.

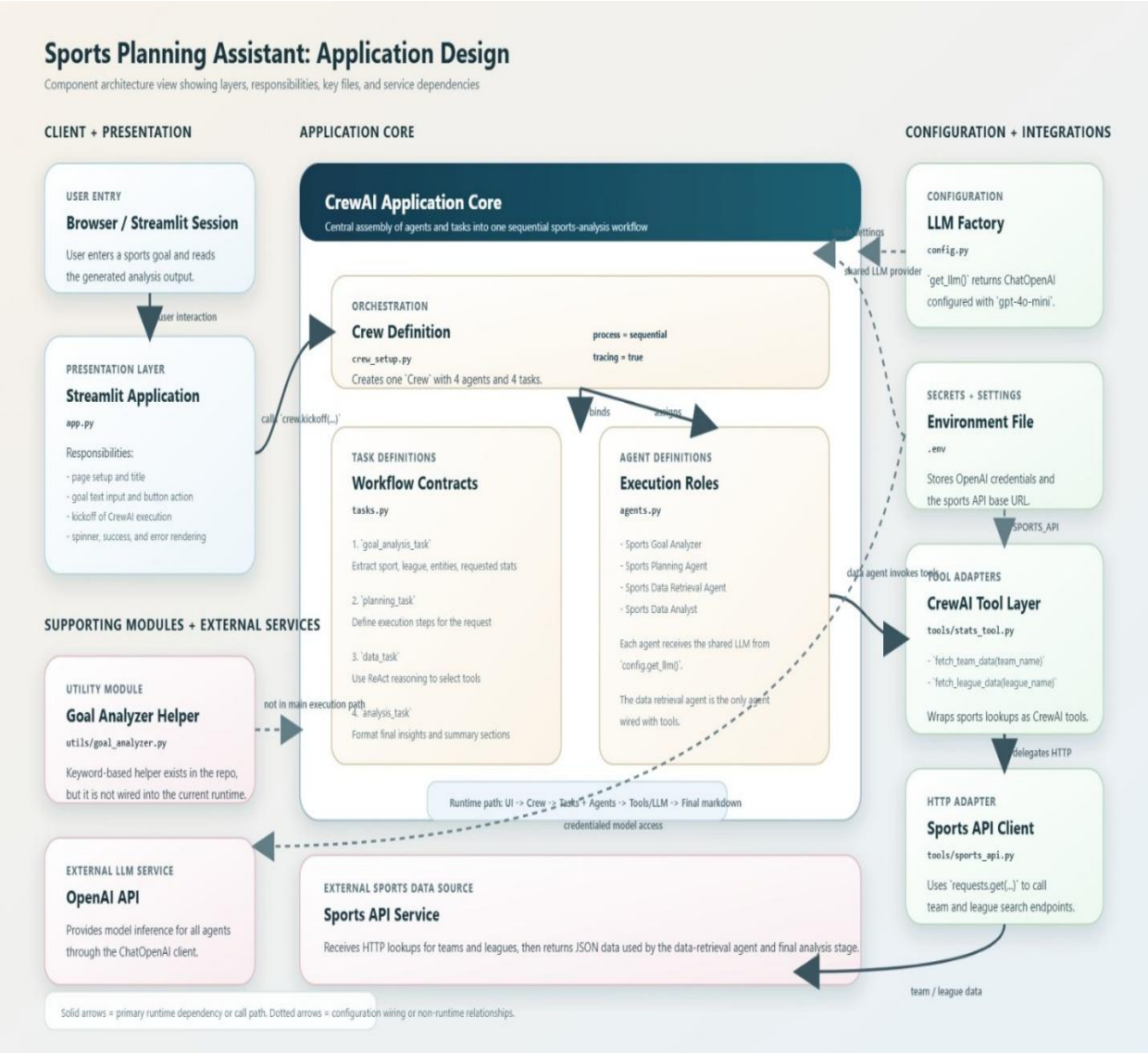
Step 7: Visualization and Output Presentation

Finally, the processed results are displayed to the user through the frontend interface. The output includes:

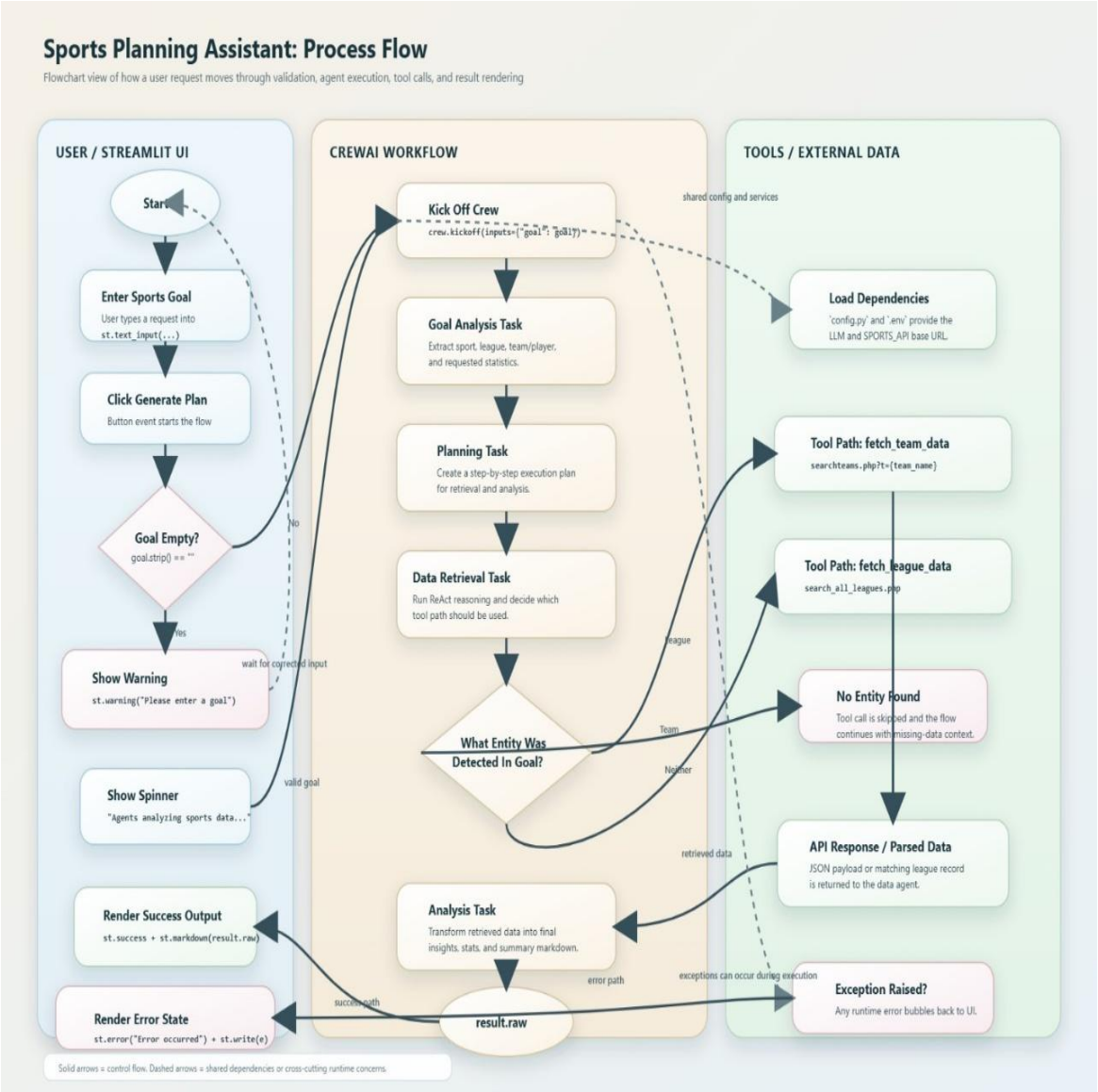
- Structured textual insights
- Graphs and charts (bar charts, pie charts, etc.)
- Tabular representation of statistics

Visualization enhances user understanding and makes the system more interactive.

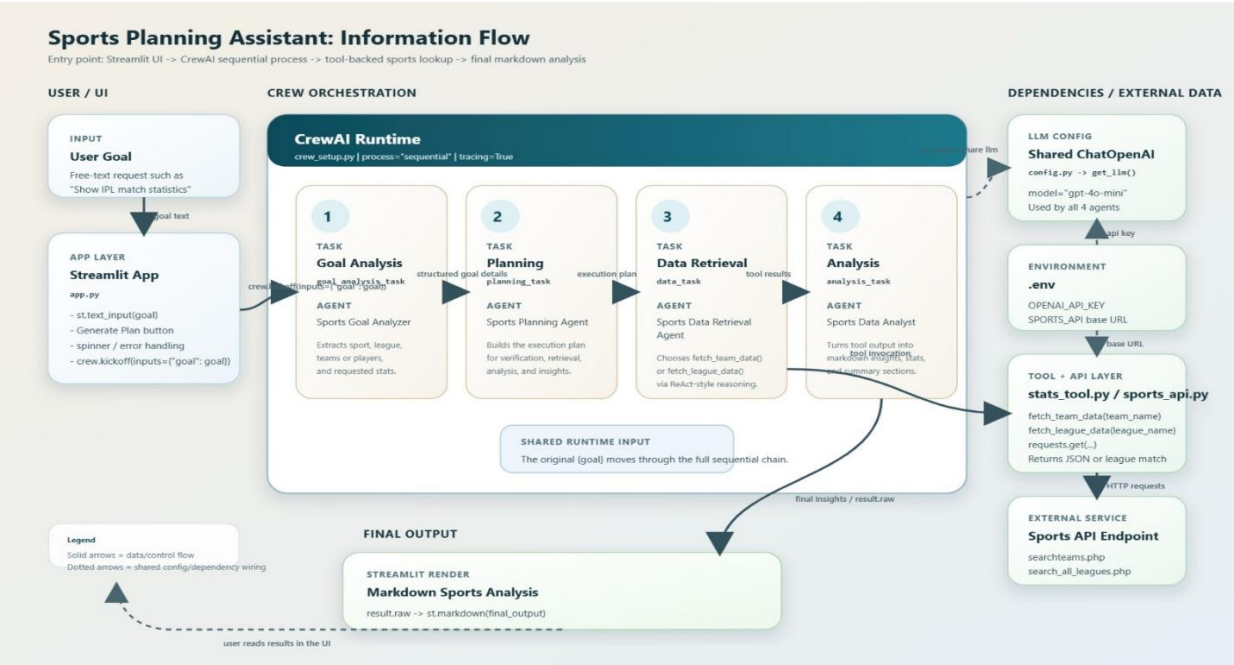
2.1Application Design



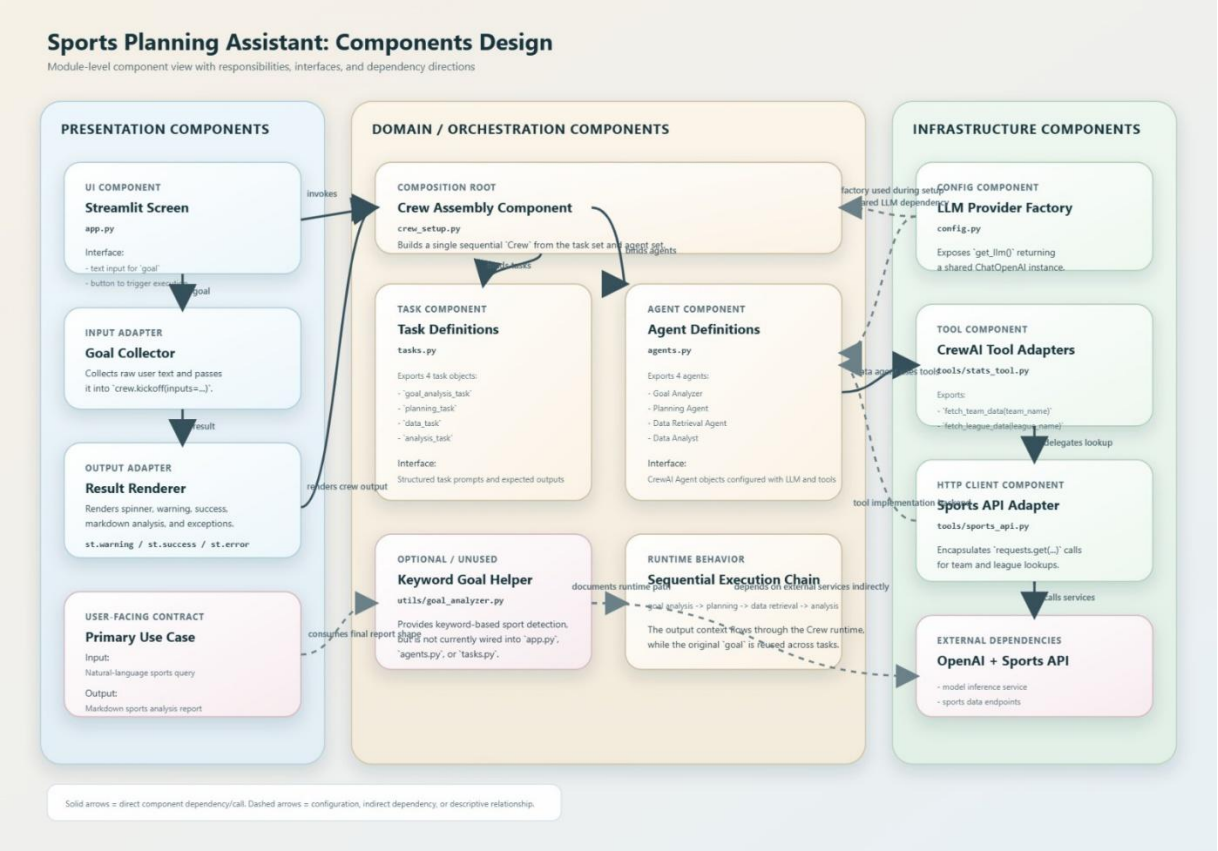
2.2 Process Flow



2.3 Information flow



2.4 Components Design



2.5 Key Design Considerations

The design of the **Sports Planning Assistant Agent** focuses on building a flexible, intelligent, and user-friendly Agentic AI system. The key considerations are as follows:

1. Multi-Sport Support
The system is designed to handle multiple sports and leagues dynamically using LLM-based goal interpretation instead of static mappings.

2. Agentic Architecture
A modular multi-agent system is used where each agent performs a specific task such as goal analysis, planning, data retrieval, and analysis.

3. ReAct Reasoning
The system follows the ReAct (Reason + Act) approach, enabling agents to think, use tools, and refine outputs dynamically.

4. Real-Time Data Integration
External sports APIs are integrated to fetch live and accurate data for teams, players, and leagues.

5. Dynamic Goal Understanding
The system automatically extracts sport, league, and intent from user input using natural language processing.

6. Modular Design
The codebase is structured into agents, tasks, tools, and frontend components, ensuring scalability and maintainability.

2.6 API Catalogue.

The system integrates external APIs to fetch real-time sports data required for analysis and planning. The primary data source used is TheSportsDB API, which supports multiple sports, leagues, teams, and players.

APIs Used

1. Search Team API

Endpoint: /searchteams.php
Purpose: Retrieves team details such as name, league, sport, and basic information.
Usage: Used when a team is identified in the user input.

2. Search League API

Endpoint: /search_all_leagues.php
Purpose: Provides a list of leagues along with associated sports.
Usage: Helps in detecting sport type from the league name.

3. Search Player API

Endpoint: /searchplayers.php
Purpose: Fetches player-related information such as team, position, and profile details.
Usage: Used when the user query involves specific players.

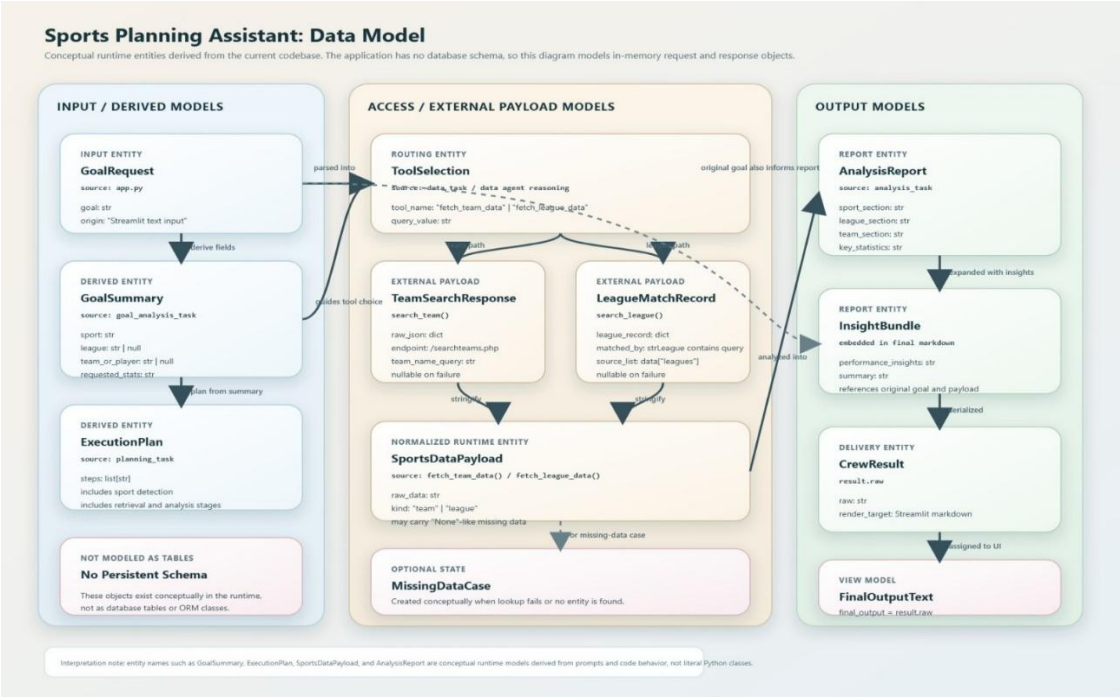
API Role in System

- Enables real-time data retrieval
- Supports multi-sport and multi-league functionality
- Used dynamically by agents through reasoning and tool selection

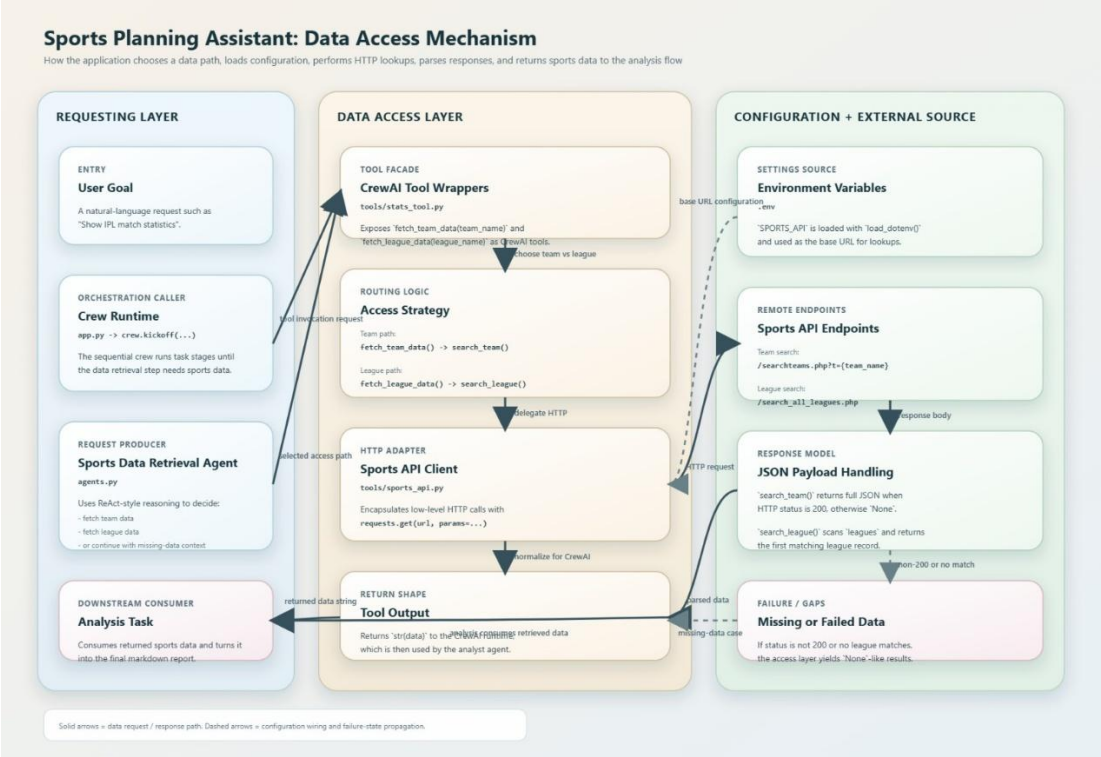
This concise API catalogue ensures efficient integration of external data into the system's workflow.

3. Data Design

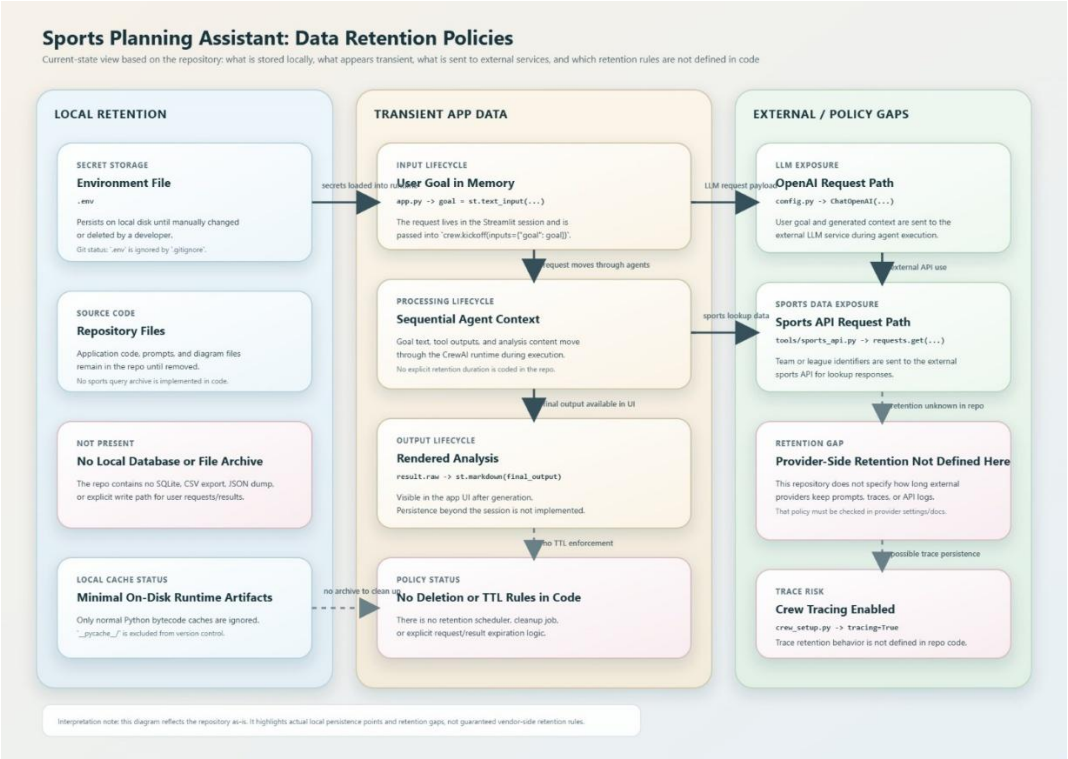
3.1 Data Model



3.2Data Access Mechanism



3.3Data Retention Policies



3.4 Data Migration

Data migration refers to the process of transferring data between storage systems, formats, or applications. In the context of the Sports Planning Assistant Agent, data migration requirements are minimal due to the system's reliance on real-time external APIs rather than internally stored datasets.

➢ **Overview**

The system does not maintain a persistent database for storing sports data. Instead, it dynamically retrieves information from external sources such as TheSportsDB API. As a result, traditional large-scale data migration processes are not required.

➢ **Data Handling Approach**

- The system follows a real-time data retrieval model
- Data is fetched on-demand based on user input
- Retrieved data is processed temporarily during runtime
- No long-term storage or data warehousing is implemented

➢ **Migration Requirements**

Since the system is designed as a lightweight and stateless application:

- No historical data migration is required
- No legacy system integration is involved
- No database schema transformation is needed

➢ **Future Considerations**

If the system is extended in the future, data migration may become relevant in the following scenarios:

- Introduction of a persistent database for storing historical sports data
- Migration from one API provider to another
- Integration with analytics platforms or data warehouses
- Implementation of caching systems with long-term storage

➢ **Key Advantages of Current Approach**

- Eliminates complexity of data migration processes
- Reduces storage and maintenance overhead

- Ensures access to up-to-date information
- Simplifies system architecture

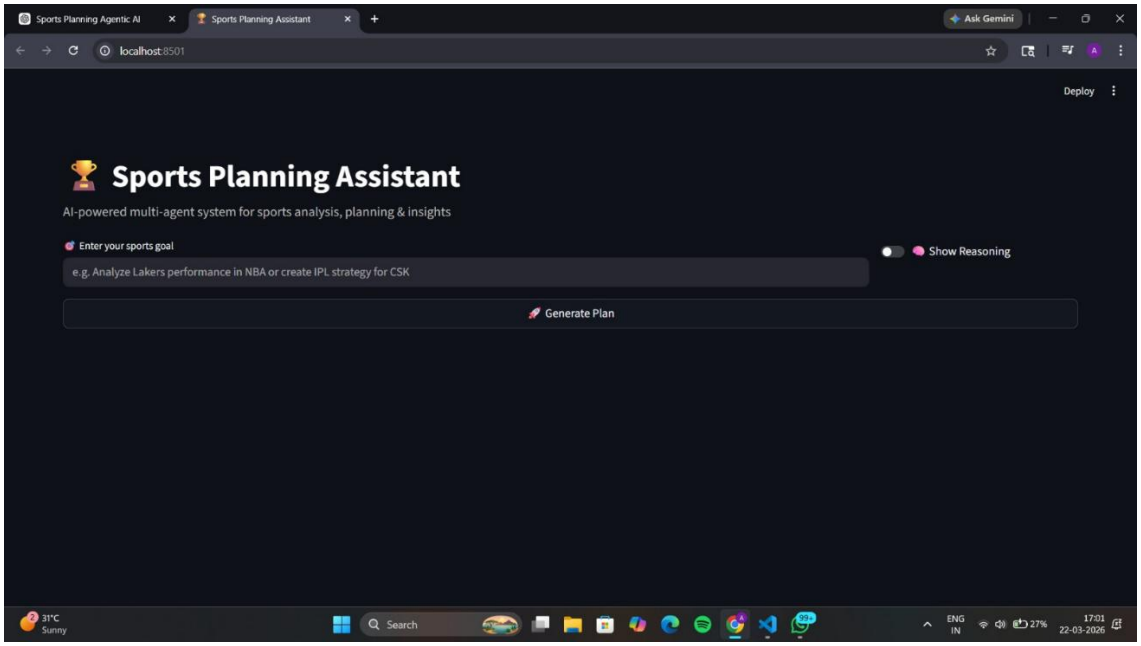
Overall, the system's reliance on real-time API-based data access removes the need for traditional data migration, making the architecture simpler and more efficient.

4. Interfaces

This section describes the interfaces between different system components including the **User Interface, LLM services, and Image Generation services**.

Types of Interfaces:

- User Interface (Web-based Frontend)
- Content Generation Interface



5. State and Session Management

The **Sports Planning Assistant Agent** manages state and session information to ensure a smooth and consistent user experience during interactions.

Session Management

- The system uses session handling provided by Streamlit to maintain user interaction within a single session.
- Each user request (goal input) is treated as an independent session task.
- Session state ensures that the user input and generated output remain available during the interaction lifecycle.

State Management

- The system maintains temporary state information such as:
  - User input (goal)
  - Intermediate agent outputs
  - Final generated results
- State is managed dynamically within the application runtime and is not permanently stored.

Stateless Backend Design

- The backend follows a **stateless architecture**, meaning:
  - No persistent storage of user data is maintained
  - Each request is processed independently
- This improves:
  - Scalability
  - Simplicity of system design
  - Data privacy

Data Flow Handling

- Input is passed sequentially across agents:
  - Goal Analyzer → Planner → Data Agent → Analysis Agent

- Intermediate results are temporarily stored and forwarded to the next stage in the workflow.

Key Characteristics

- Lightweight session handling
- No long-term data storage
- Secure and privacy-friendly design
- Efficient handling of real-time requests

This approach ensures that the system remains **efficient, scalable, and secure** while processing user requests in real time.

6. Caching

Caching is an important non-functional aspect of the **Sports Planning Assistant Agent**, aimed at improving system performance, reducing API latency, and minimizing redundant computations. Since the system relies on external APIs and multi-step agent reasoning, caching helps in optimizing response time and resource usage.

Caching Strategy

The system primarily uses **in-memory caching techniques** during runtime. Since the backend is designed to be lightweight and stateless, caching is implemented at the application level rather than using a dedicated caching server.

Caching can be applied at multiple stages:

1. API Response Caching

- Responses received from external APIs such as TheSportsDB API can be temporarily stored.
- If the same team, league, or player data is requested again, the system retrieves it from cache instead of making a new API call.
- This significantly reduces network overhead and latency.

2. Agent Output Caching

- Intermediate outputs generated by agents (e.g., detected sport, league, or team) can be cached during a session.
- This prevents repeated processing of the same input within the same workflow.

3. Visualization Data Caching

- Processed data used for generating charts and graphs can be cached to avoid recomputation when rendering visualizations multiple times.

Implementation Approach

- Caching is implemented using built-in mechanisms available in Streamlit such as:
  - st.cache_data() for storing API responses
  - st.session_state for maintaining temporary session data
- Cached data is stored in memory and is cleared automatically when:
  - The application restarts
  - The session expires

Cache Invalidation

To maintain data accuracy, the system follows simple cache invalidation strategies:

- Cached data is refreshed periodically to ensure up-to-date information
- In cases where real-time accuracy is critical, fresh API calls are preferred over cached data
- Cache is cleared when session ends to avoid stale data usage

7. Non-Functional Requirements

Non-functional requirements define the quality attributes of the **Sports Planning Assistant Agent**, ensuring that the system performs efficiently, securely, and reliably while handling user requests.

7.1 Security Aspects

The system incorporates basic security measures to ensure safe handling of data and API interactions.

- **API Key Protection:** Sensitive credentials (e.g., API keys) are stored securely using environment variables and are not exposed in the source code.
- **Input Validation:** User inputs are validated to prevent malformed queries or unintended processing.
- **Secure API Communication:** All external API calls are made over secure protocols (HTTPS) to ensure data integrity.

- **Limited Data Exposure:** The system only retrieves and processes publicly available sports data, minimizing risk.
- **Error Handling:** Controlled error messages are displayed to avoid leaking internal system details.

7.2 Performance Aspects

- The system is designed to provide fast and efficient responses while handling multiple operations such as reasoning, API calls, and data visualization.
- **Efficient Agent Workflow:** The multi-agent architecture ensures that tasks are executed in a structured and optimized sequence.
- **Dynamic Tool Usage:** The ReAct-based reasoning mechanism ensures that only necessary API calls are made, reducing latency.
- **Lightweight Frontend:** The interface built using Streamlit ensures quick loading and smooth interaction.
- **Optimized API Calls:** Minimal and targeted API requests are used to reduce response time.
- **Scalability:** The modular architecture allows future enhancements such as caching or parallel processing to improve performance.

  This ensures that the system remains **secure, responsive, and scalable** while delivering real-time sports analytics.

8. References

The following resources and technologies were used in the design and development of the Sports Planning Assistant Agent:

1. CrewAI – Used for implementing the multi-agent architecture and task orchestration.
2. LangChain – Used for integrating language models and managing agent workflows.
3. Streamlit – Used for developing the user interface and visualization dashboard.
4. TheSportsDB API – Used for retrieving real-time sports data including teams, leagues, and players.
5. Agentic AI – Conceptual foundation for designing autonomous multi-agent systems.
6. ReAct framework – Used for implementing dynamic reasoning and tool usage by agents.
7. Official documentation of Python libraries and frameworks used in development.